

MSA

co2plant

2025.10.22

CONTENTS

CHAPTER 01

모놀리스, 모듈러 모놀리스, MSA?

CHAPTER 02

MSA의 핵심 원칙

CHAPTER 03

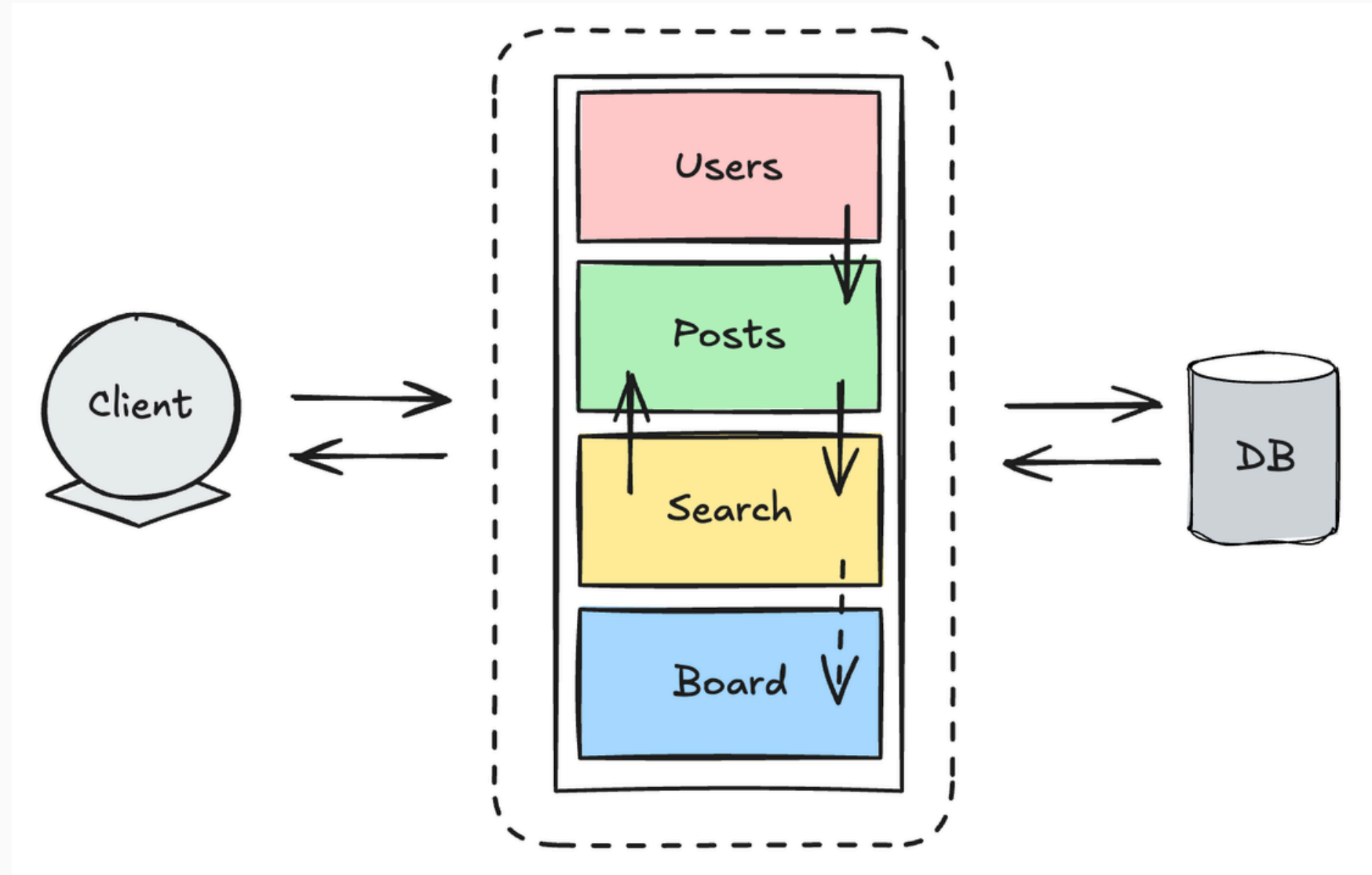
MSA의 장점과 단점

CHAPTER 04

언제 선택하는가

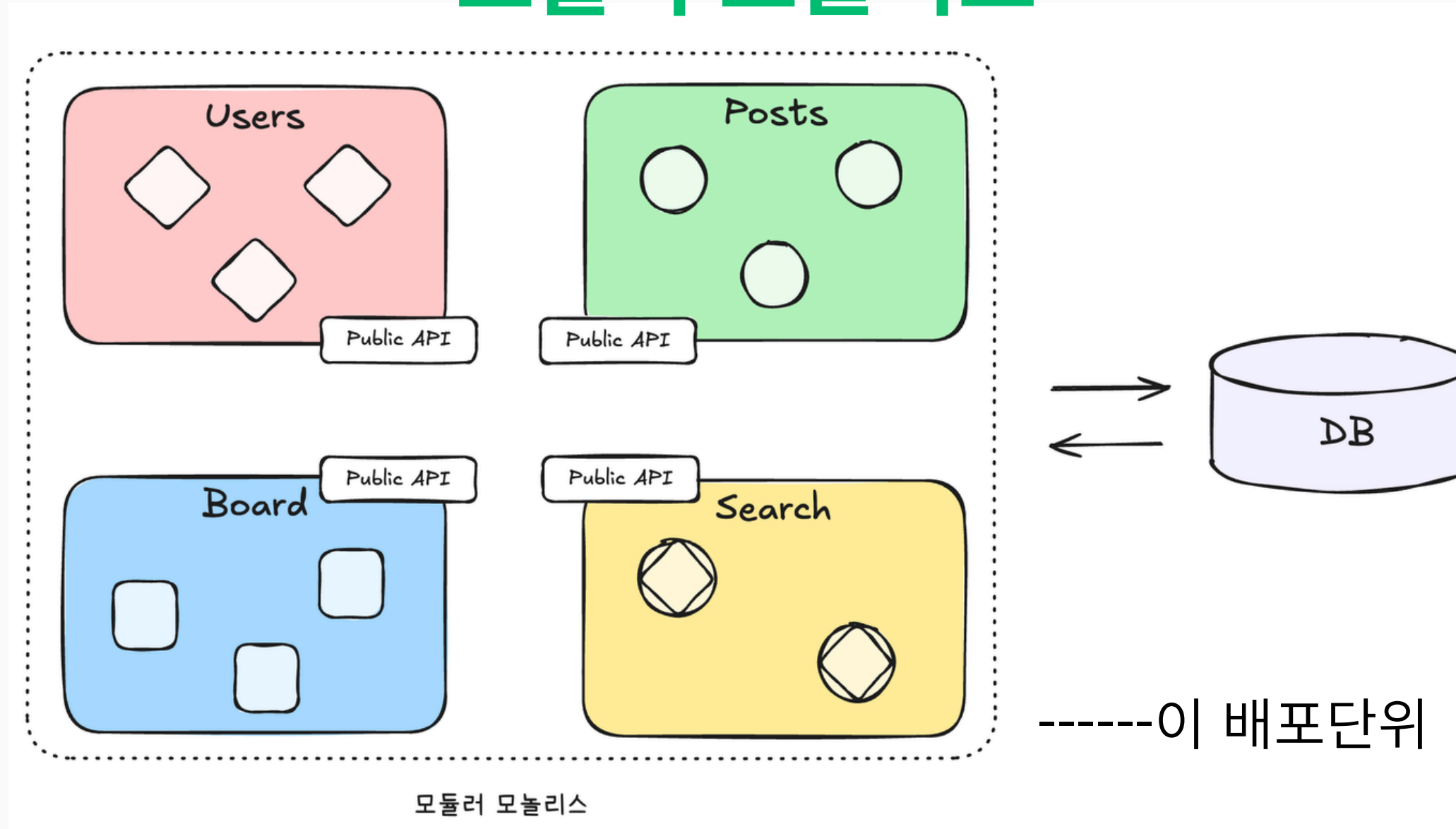
모놀리스, 모듈러 모놀리스, MSA?

모놀리스(모놀리식)



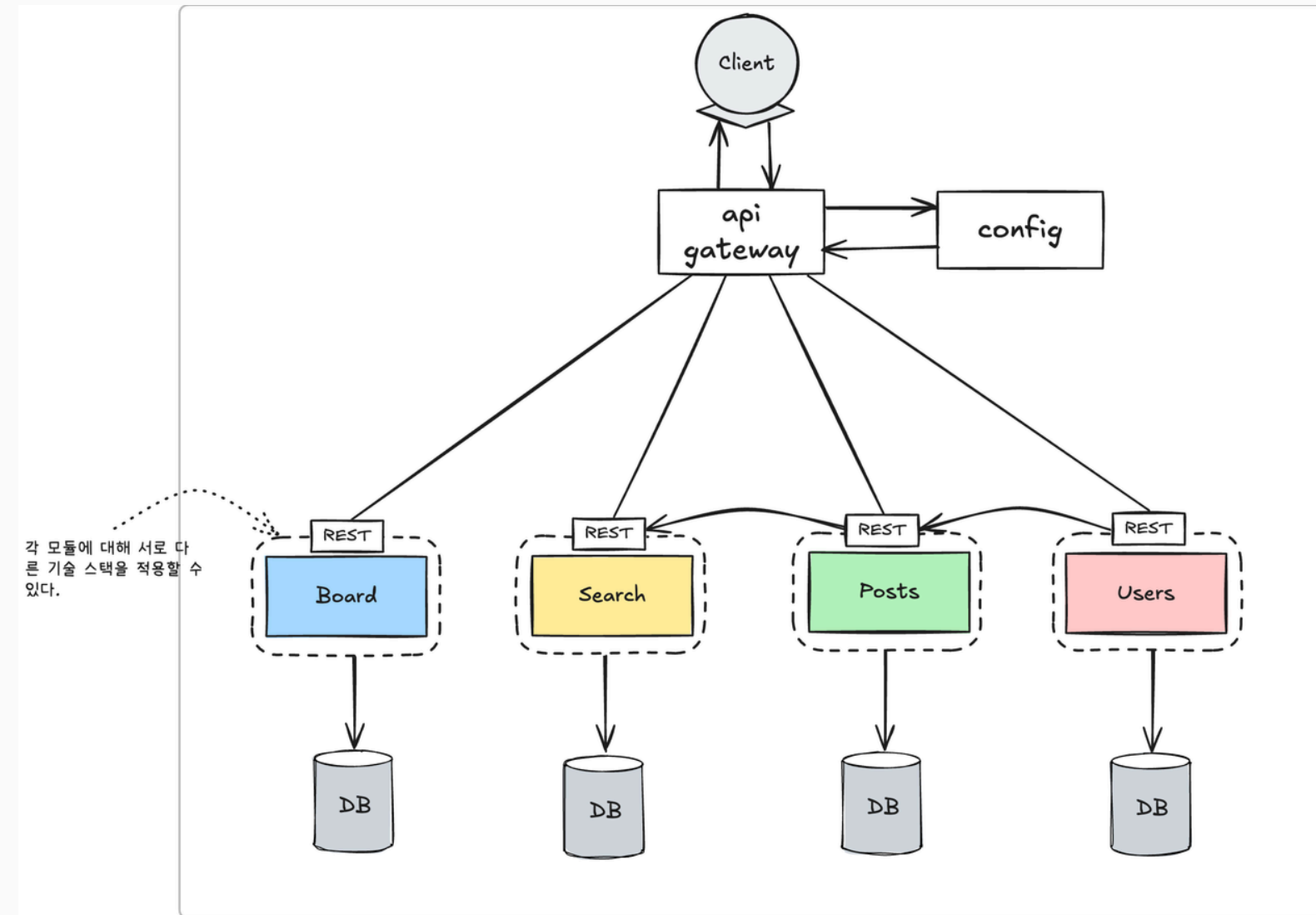
모든 기능이 **단일 코드베이스**, **단일 프로세스**,
단일 배포 단위로 구성되는 방식

모듈러 모놀리스



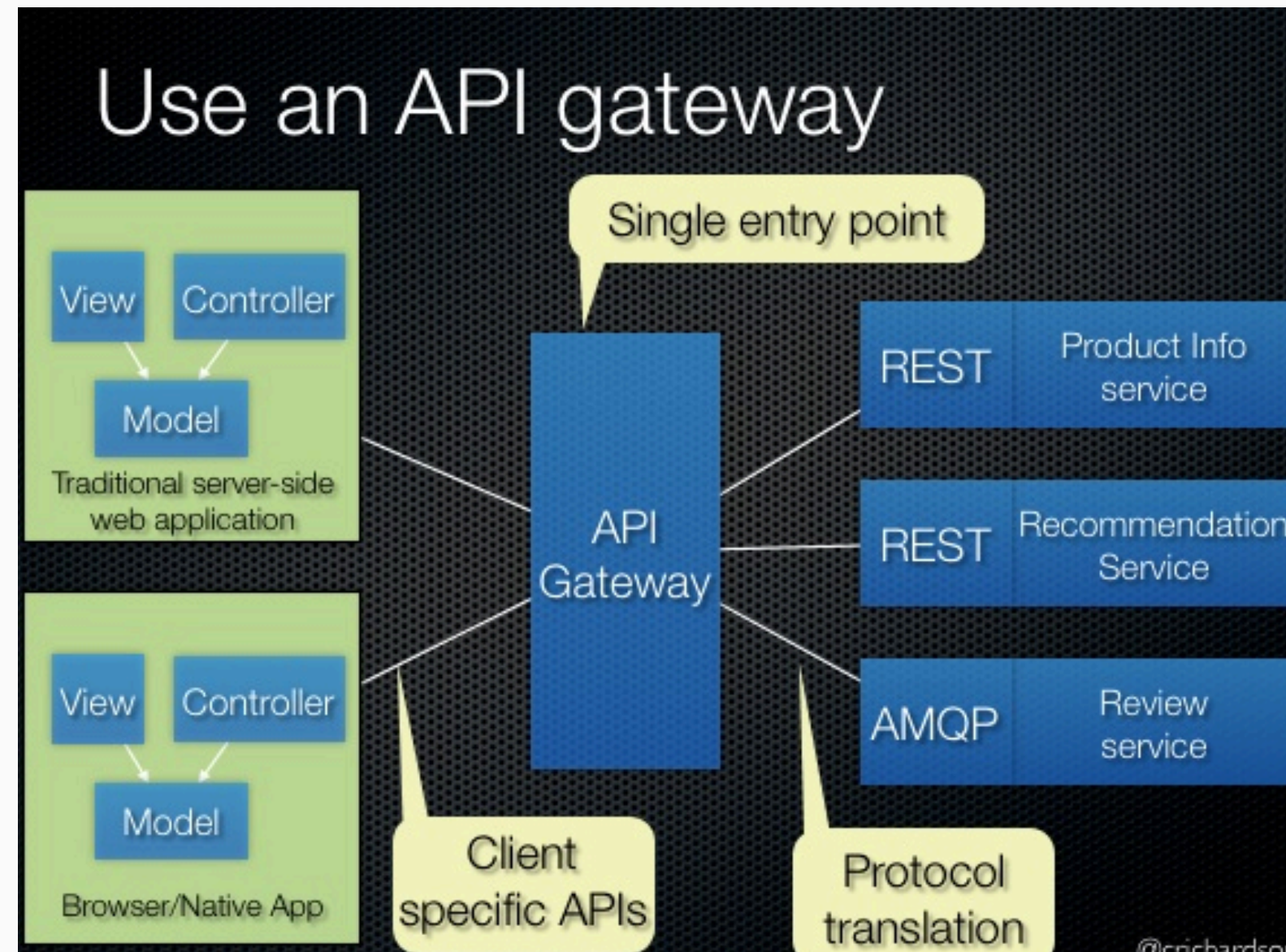
독립적인 모듈로 구성되어 있지만 **단일 코드베이스**,
단일 배포 단위와 **단일 런타임 환경** 내에서 운영되는 방식

Microservice Architecture



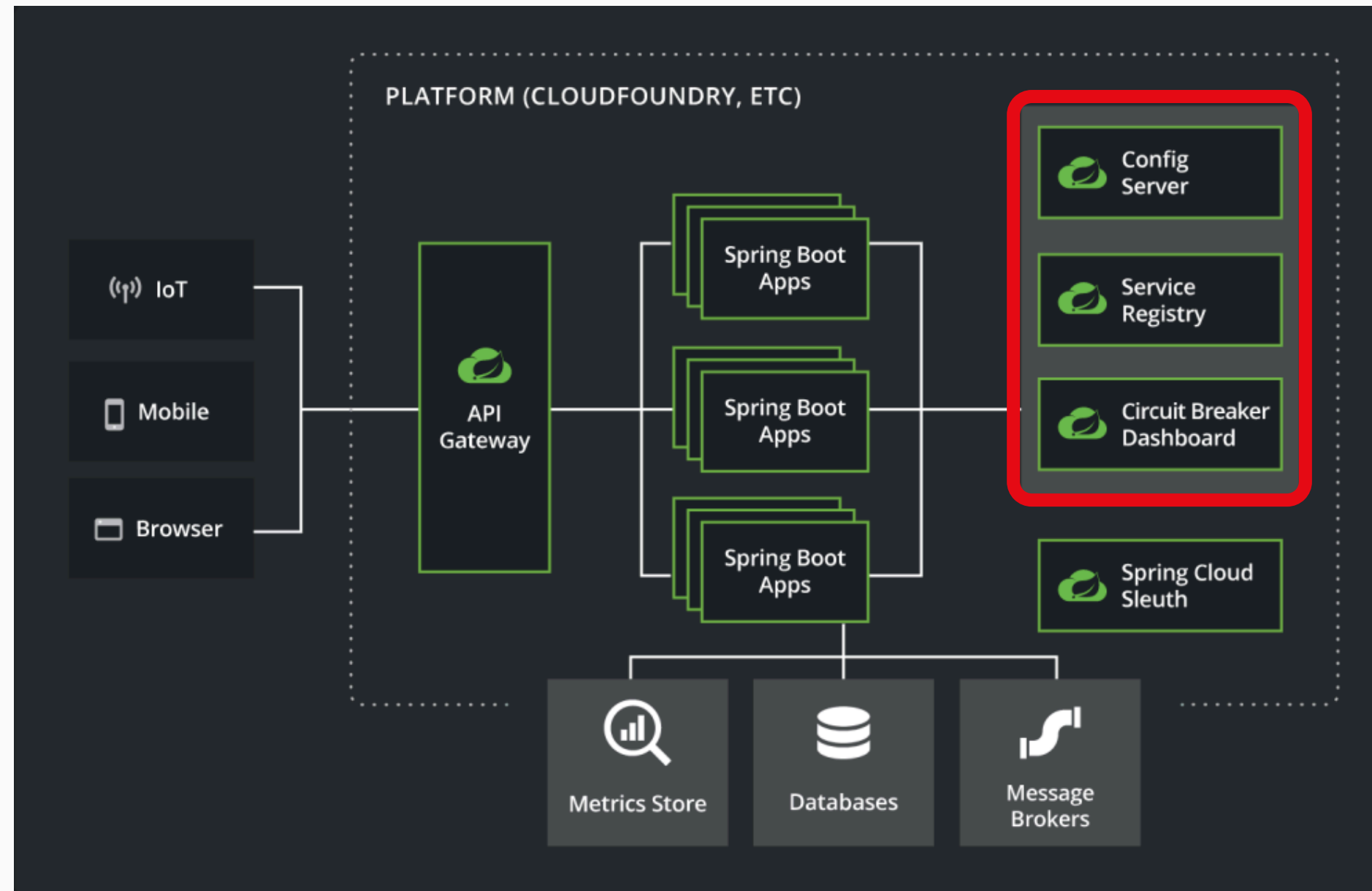
하나의 큰 애플리케이션을
여러 개의 작은 독립 서비스들로 나누어 구축하는 방식

API Gateway 패턴



클라이언트가 서비스에 접근하는 단일 진입점 역할
내부 서비스가 추상화됨

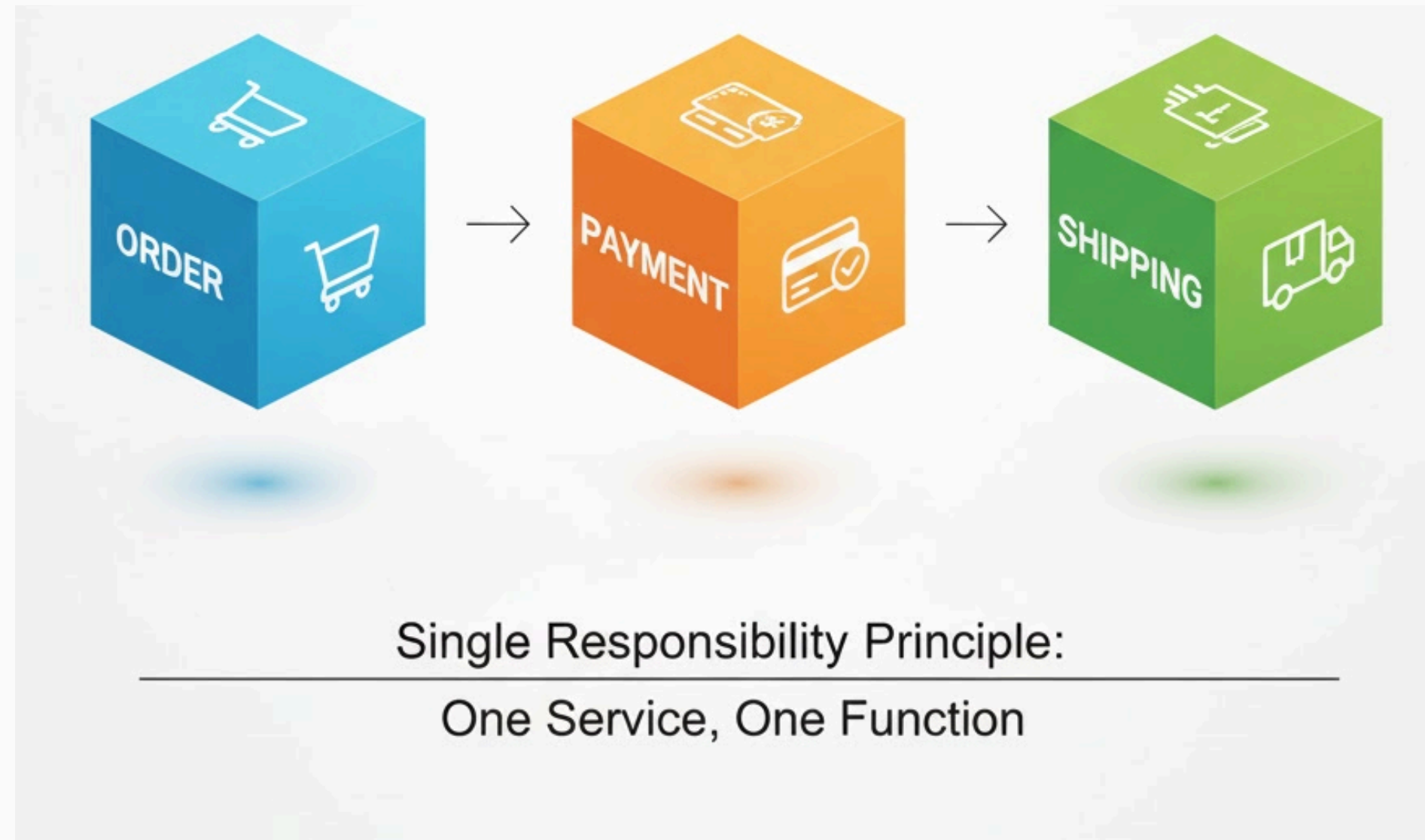
서비스 디스커버리



인스턴스 들이 **동적으로 생성, 변경, 소멸**됨
서비스 인스턴스의 **위치와 상태를 추적**

MSA 핵심 원칙

단일 책임의 원칙



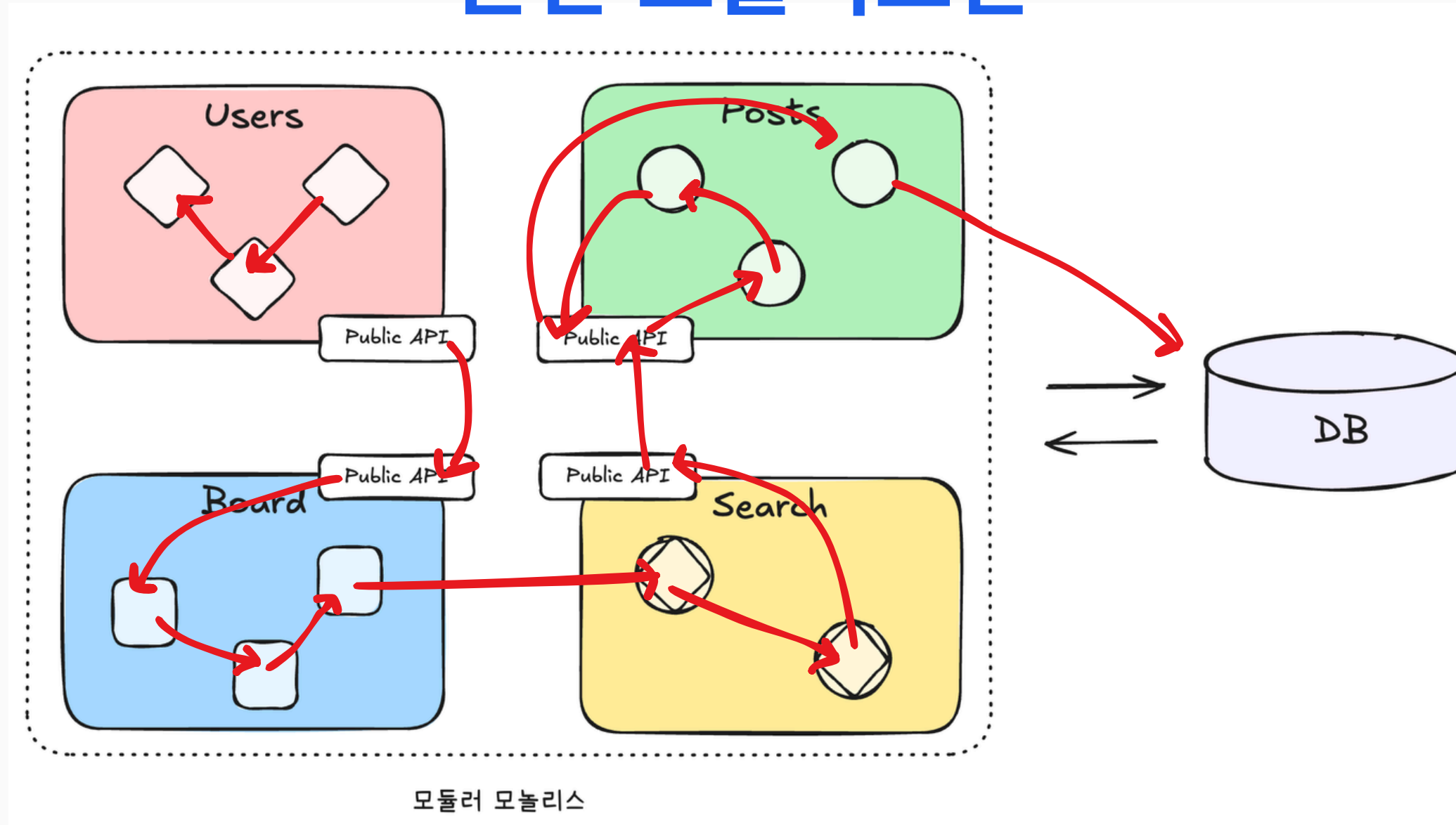
각 서비스는 다른 서비스의 기능을 포함해서는 안됨
서비스를 변경할 때, **해당 서비스에 대한
비즈니스 책임 영역, 부서는 오직 하나**여야 함

느슨한 결합



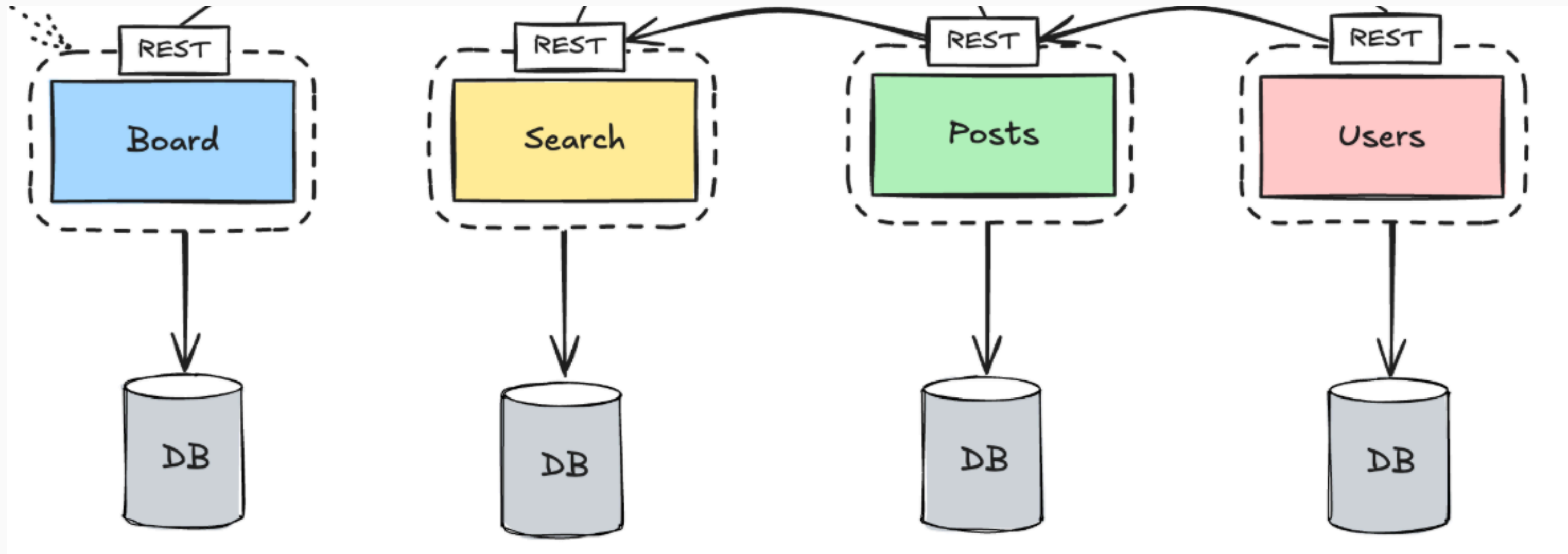
하나의 비즈니스 역량에 필요한 모든 로직, 데이터를
서비스 내부에 캡슐화 하도록 보장
단, 동기식 호출 의존성이 많을 수록 **분산 모놀리스**에 가까워짐

분산 모놀리스란



MSA의 장점(독립성, 확장성)은 누리지 못하고,
단점(네트워크 복잡성, 분산 트랜잭션)만 모두 가져가는 아키텍처

분산 데이터 관리



각자 자신만의 **DB를 소유, 관리**하며
다른 서비스는 이 DB에 접근할 수 없고,
해당 서비스가 제공하는API를 통해서 접근가능

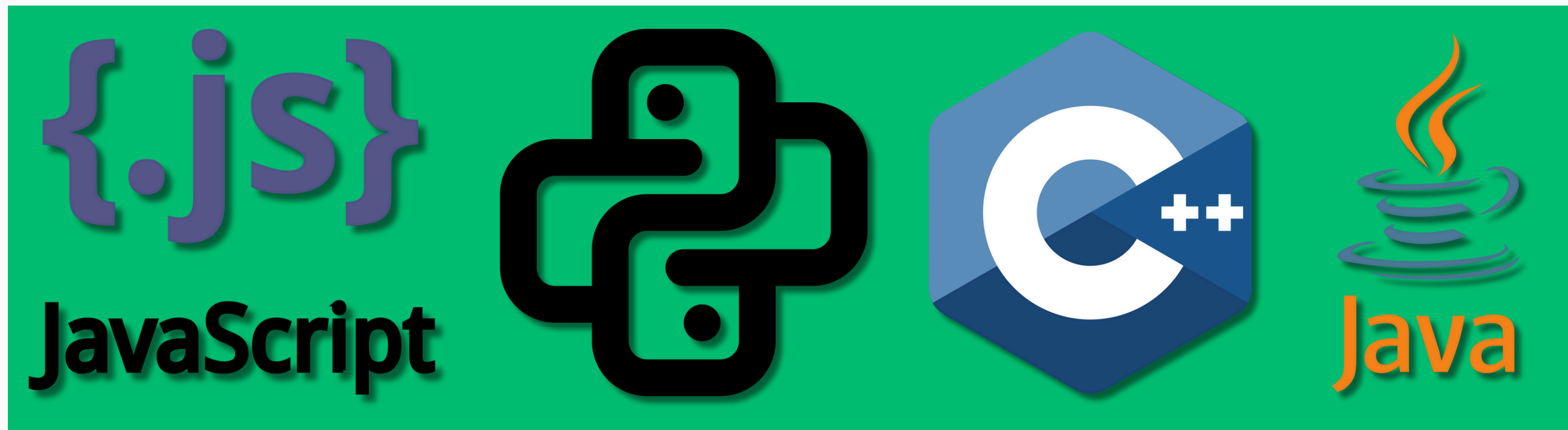
도메인 주도 설계



비즈니스 능력에 맞춰 서비스를 구성
응집력을 향상, 서비스간 종속성 감소

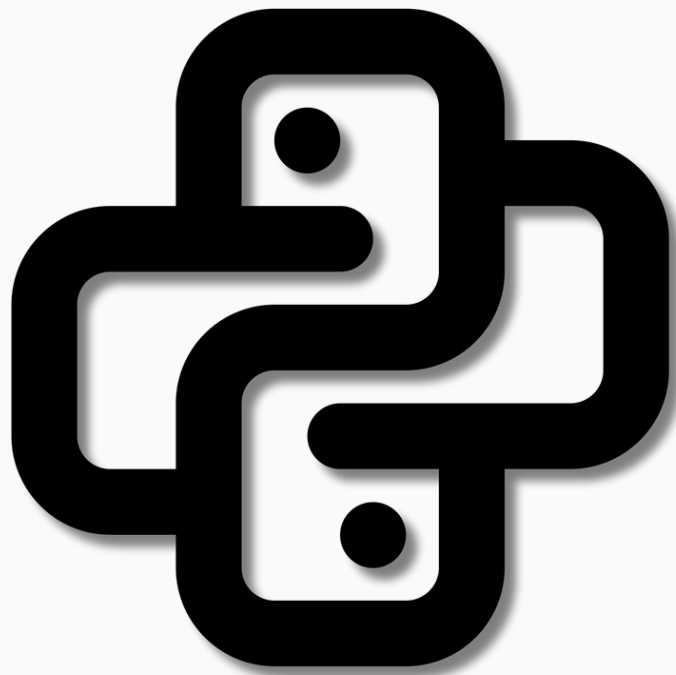
MSA 주요 특징 및 장점

다양한 기술 스택



각 서비스들은 API를 통해 통신하기 때문에
언어에 비종속적임

만약 AI서비스와 고성능 백엔드가 필요하다



AI서비스는 Python
고성능 백엔드 서비스는 Java로 만들어
하나의 시스템을 만들 수 있음

독립적인 서비스 개발, 배포



Board



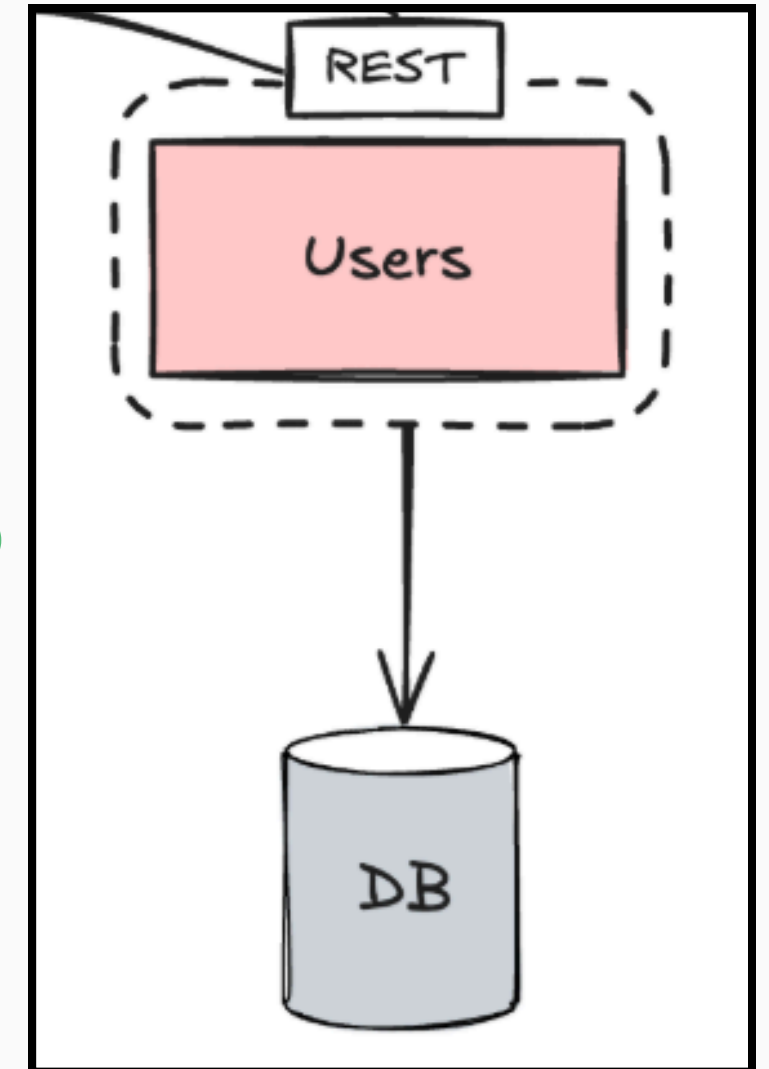
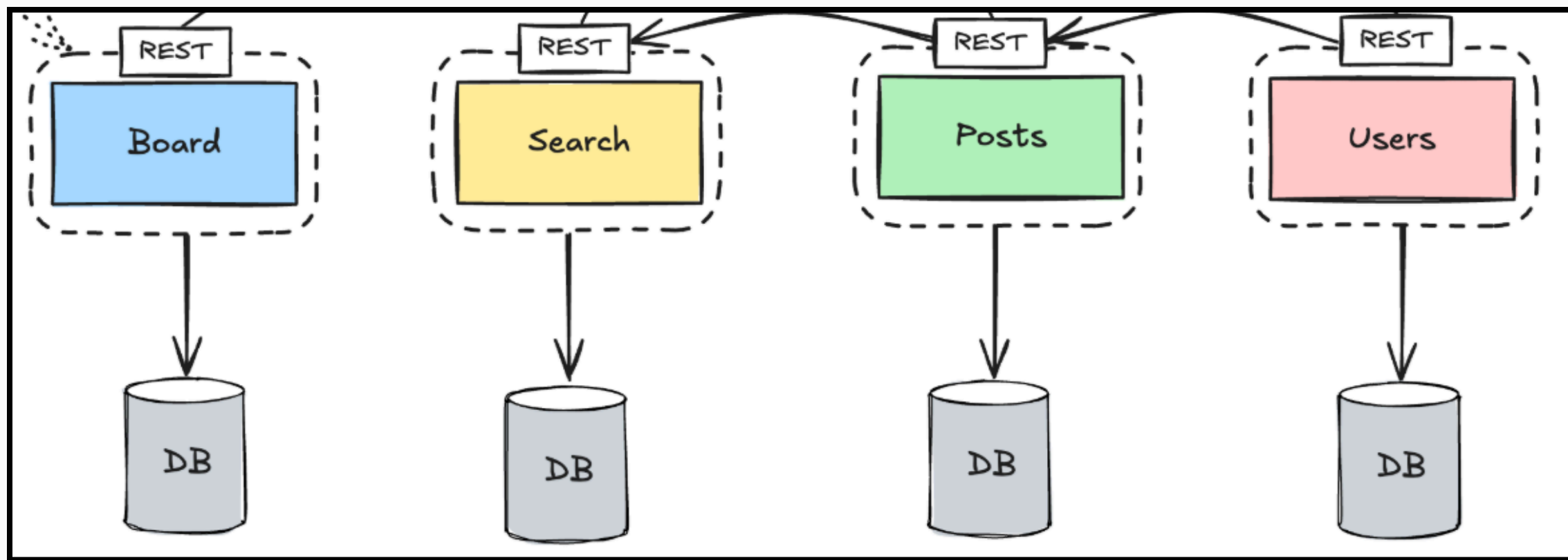
Search



User

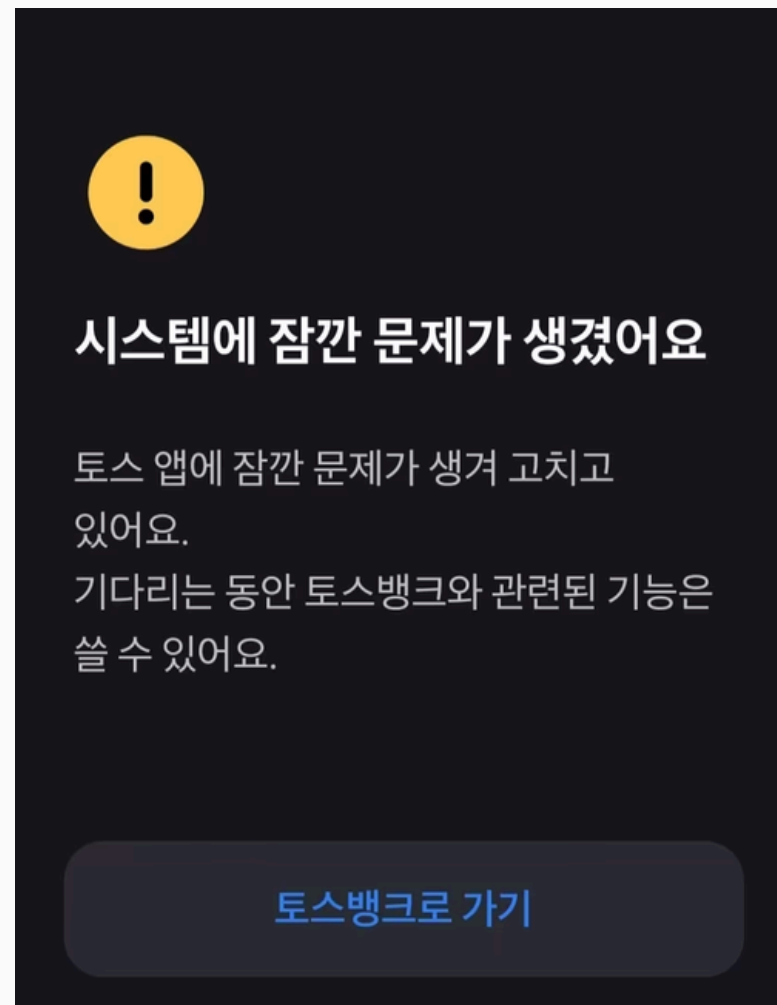
특정 서비스가 변경되더라도
전체 시스템을 중단하고 재배포할 필요가 없음
더 빠르고 자주, 안정적으로 배포 가능

효율적인 확장성



시스템 전체가 아닌, 필요한 서비스만
선택적으로 수평적, 수직적 확장

장애 격리



하나의 서비스에서 장애가 발생하더라도
시스템 전체가 중단되지 않음
복원력 향상, 안정성 향상

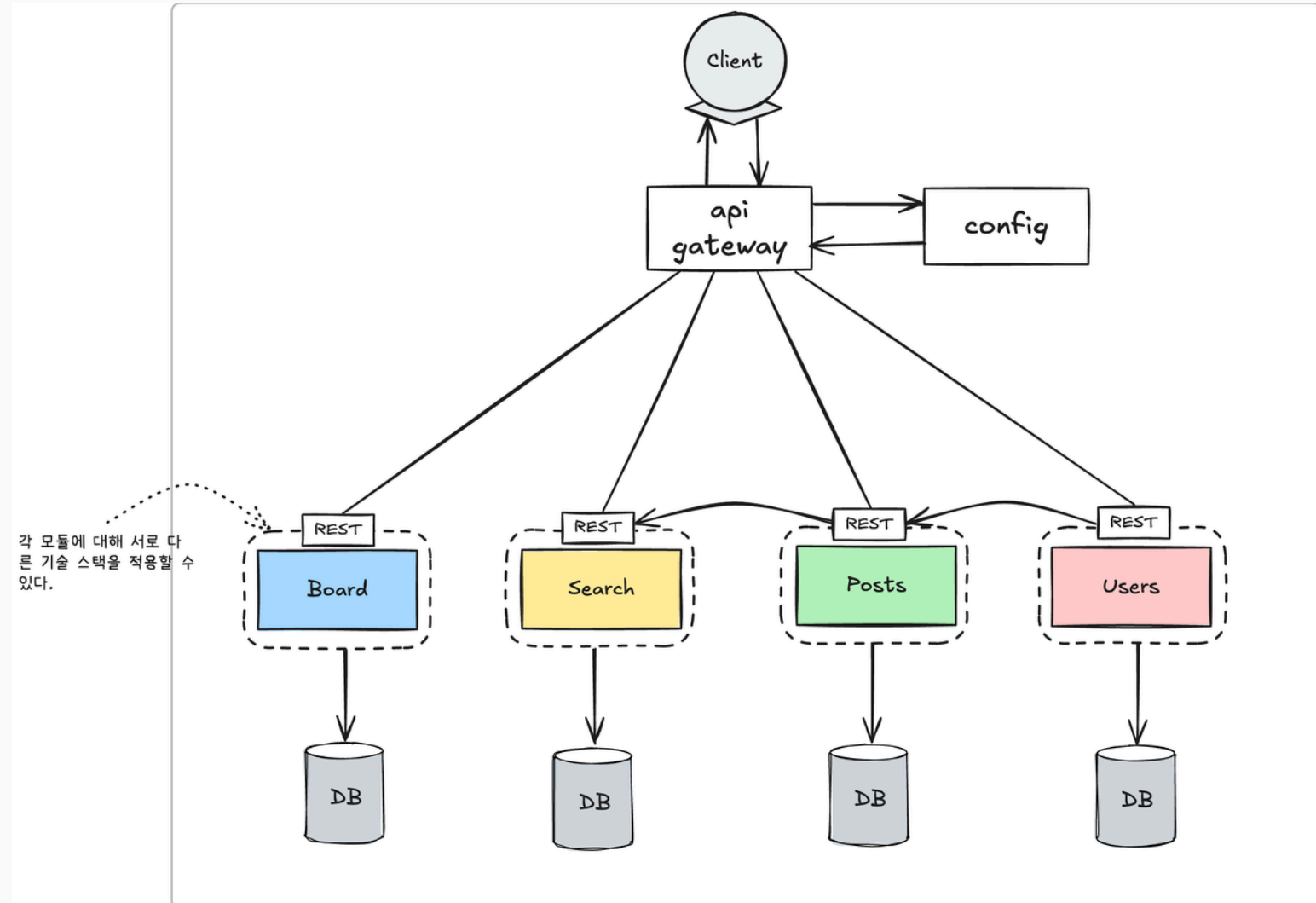
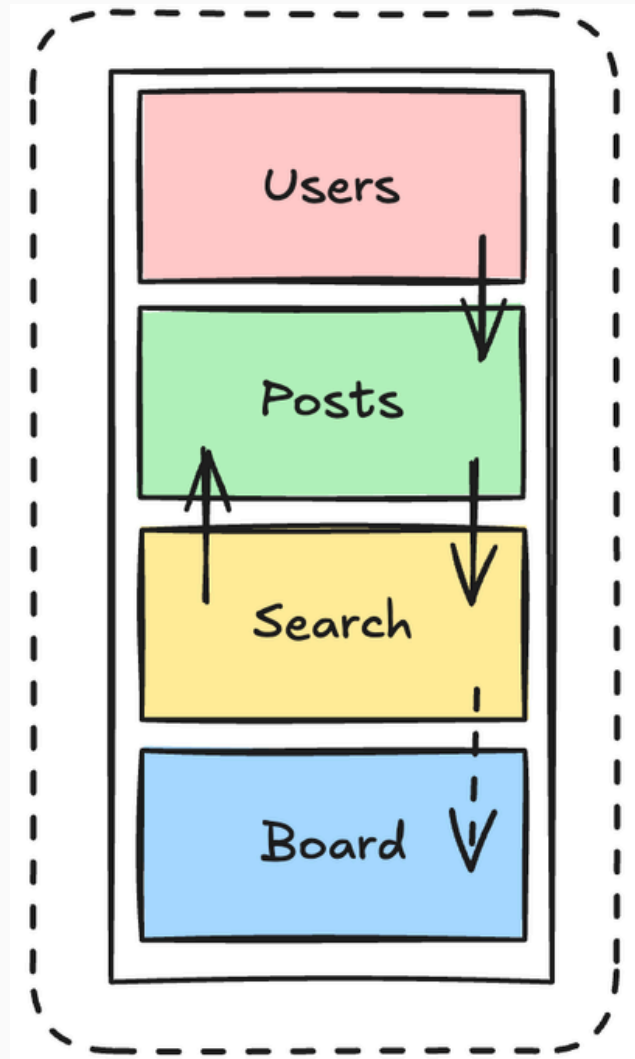
컨테이너 친화적



특유의 아키텍처 구조가 **컨테이너 기반의 플랫폼과 잘 어울려**서 과거보다 이용하기 쉬워짐

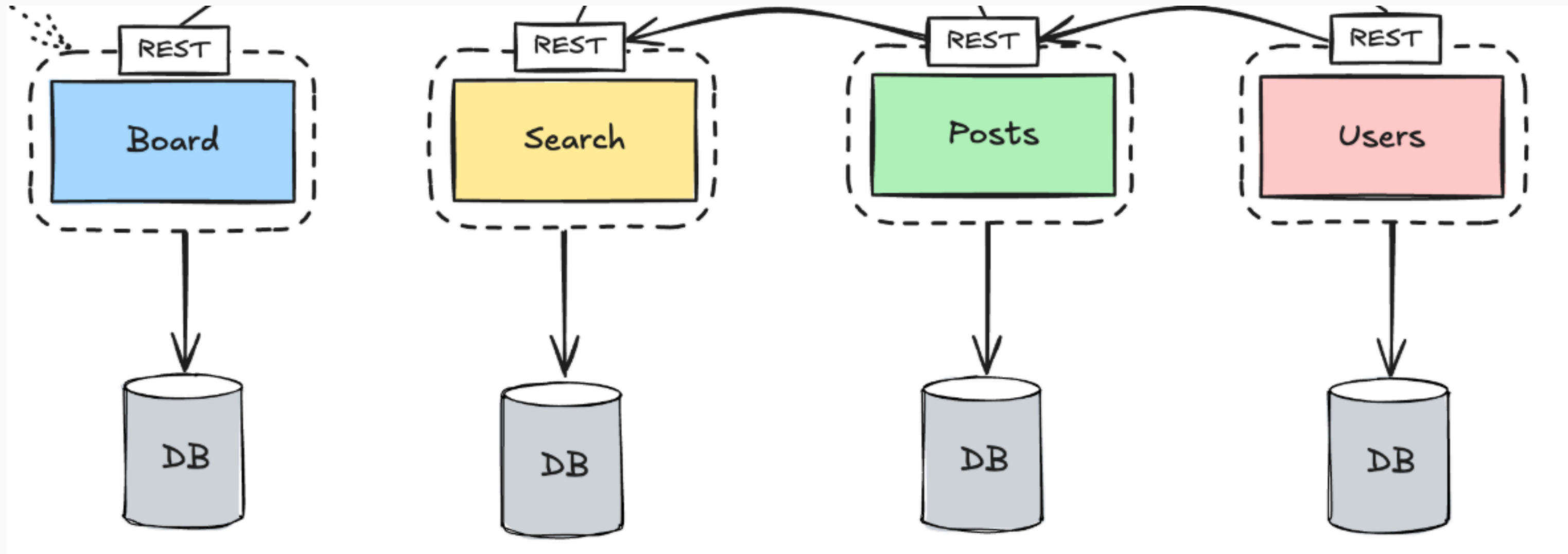
단점

높은 개발 복잡도



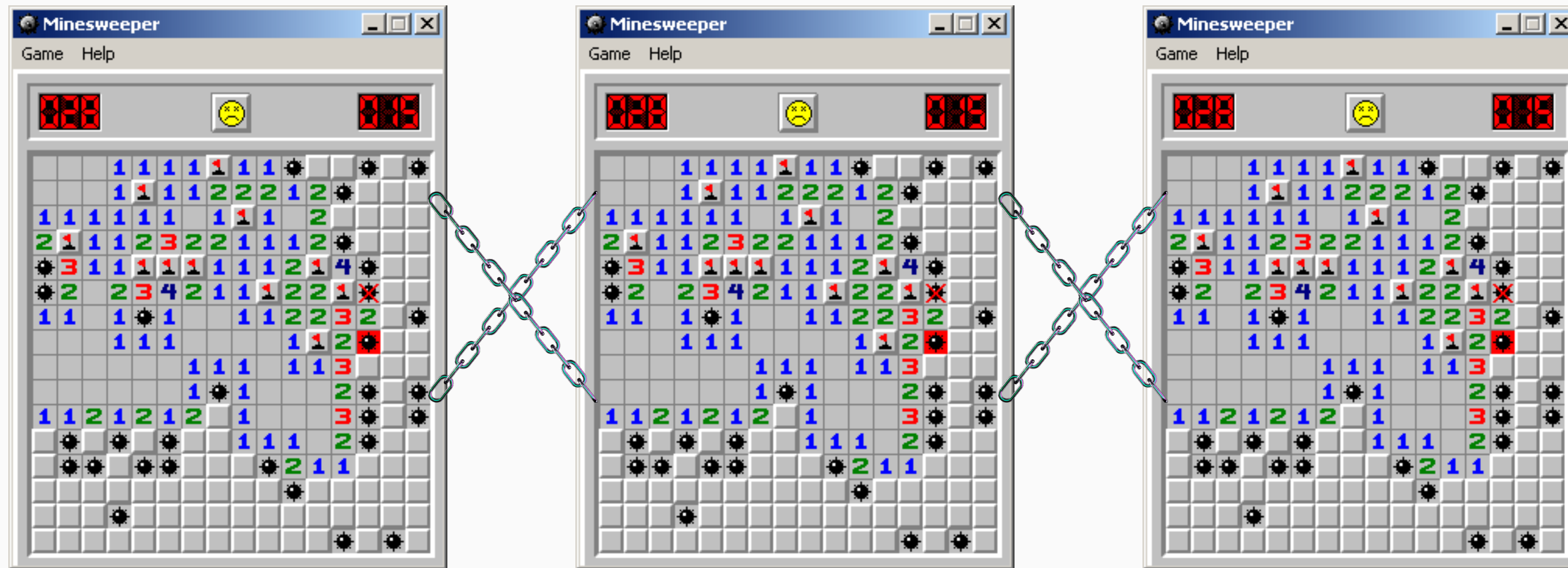
모놀리스의 메모리 내 호출이 **네트워크 호출로 변경**되기 때문에
네트워크 장애 내성 설계는 물론 네트워크 지연 시 **연쇄 장애를 방지**해야함

분산 데이터 관리



각 서비스는 자신만의 독립적인 DB를 소유하기 때문에
분산 트랜잭션이 필요해짐. 또한 중간 과정 실패 시
데이터가 **일시적으로 불일치 상태**에 빠짐

통합 테스트의 복잡성



하나의 기능 테스트를 위해, 의존 관계에 있는 **모든 서비스를
함께 실행** 해야함 또한, 여러 서비스를 거치는
기능의 경우 **어디서 오류가 발생했는지 찾기 어려워짐**

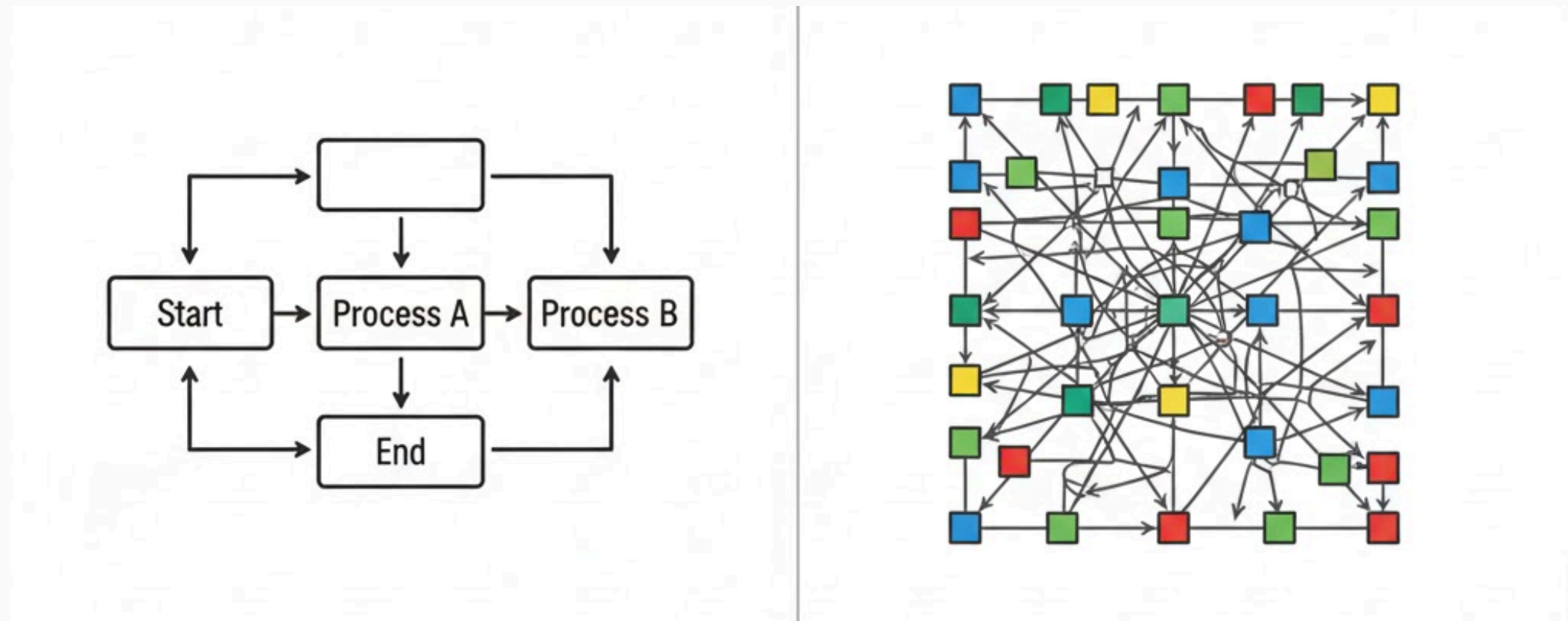
배포 및 운영 복잡성 증가



관리 해야할 배포 단위가 **1개에서 N개로** 증가
N개의 서비스를 개별적으로 **모니터링, 로깅, 알람, 설정관리** 해야함
수동배포는 거의 불가능해서 **컨테이너가 필수**

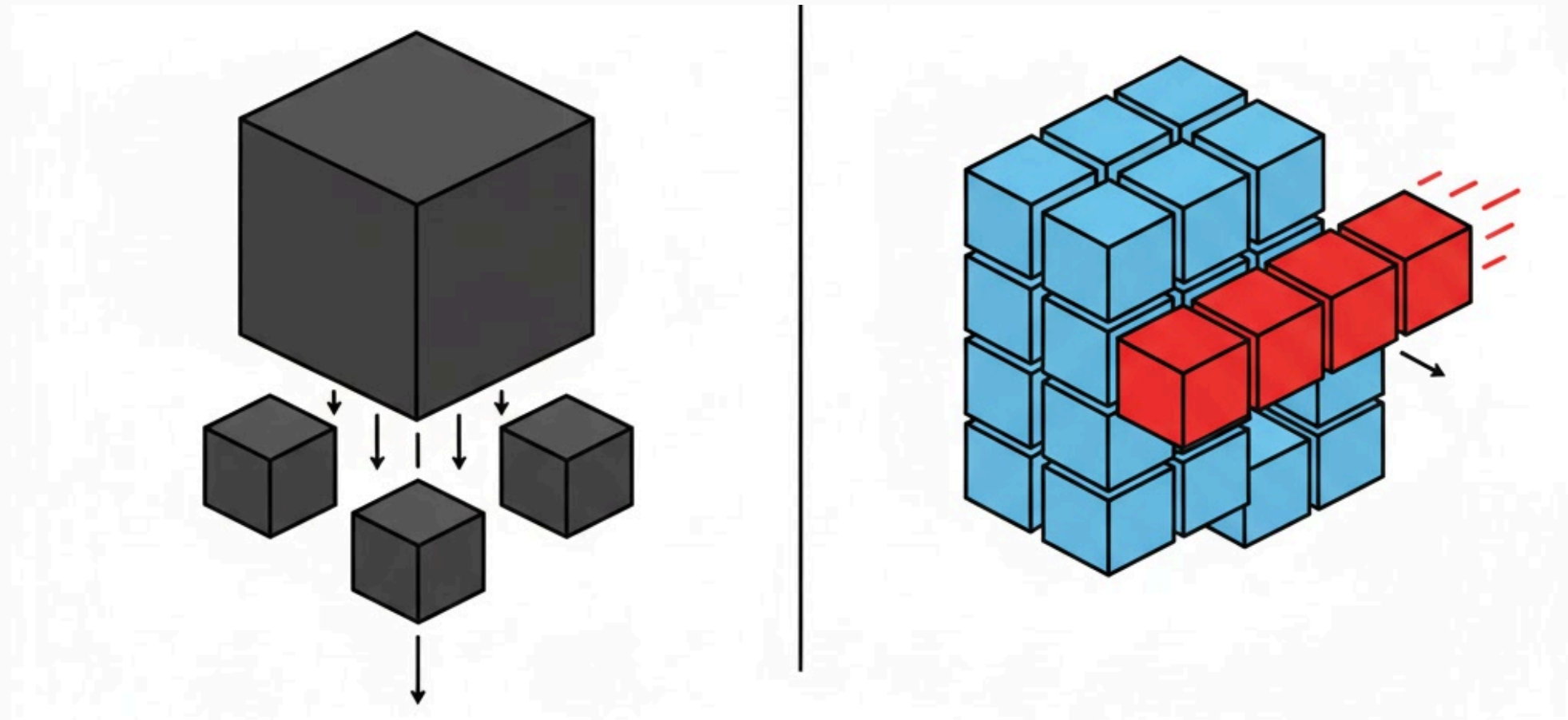
MSA는 언제 적용하는가

비즈니스 복잡도에 따른 결정



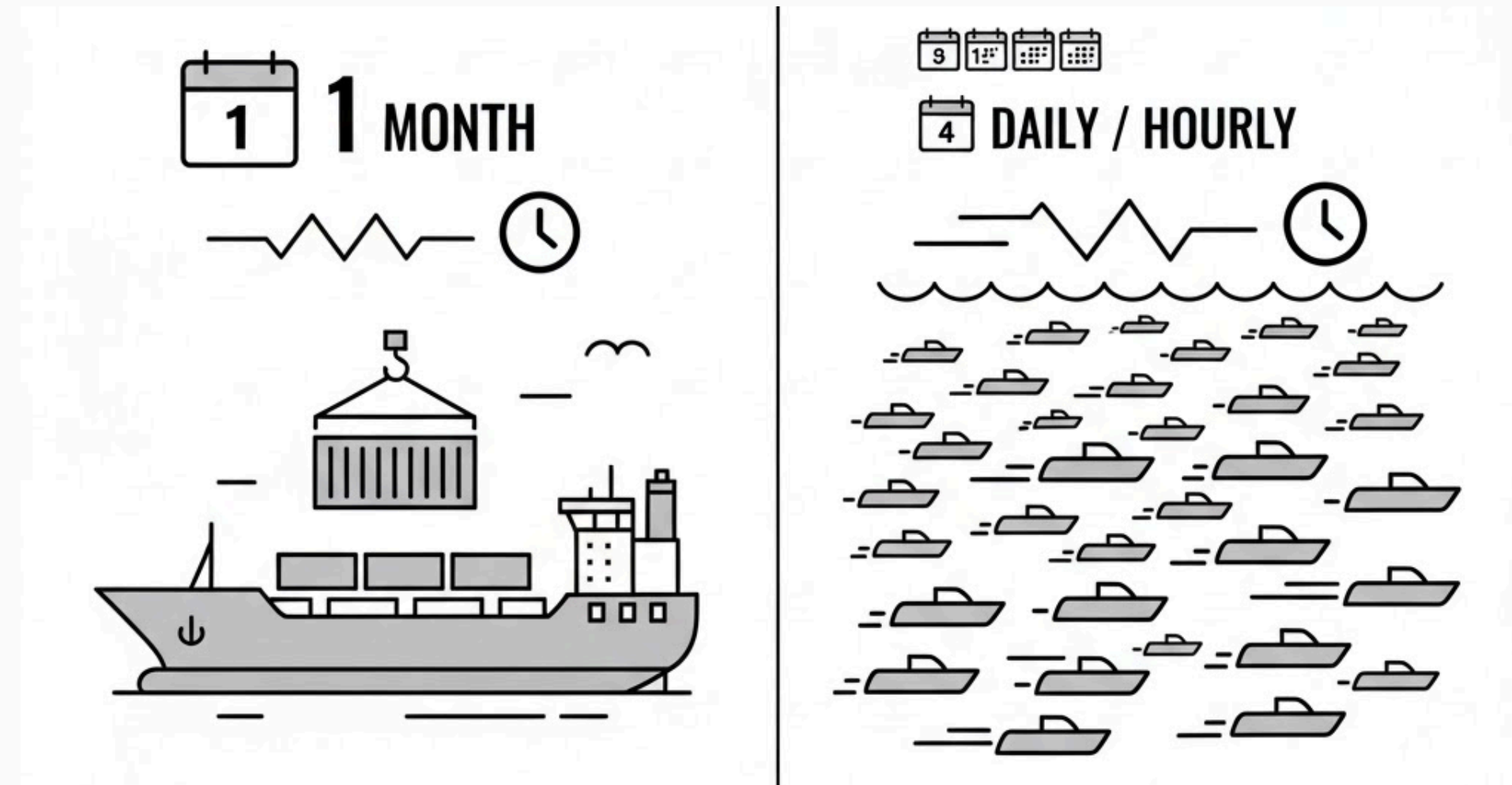
단순하고 안정적인 도메인 : 모놀리스
복잡하고 빠르게 변하는 도메인 : MSA

확장 요구사항에 따른 결정



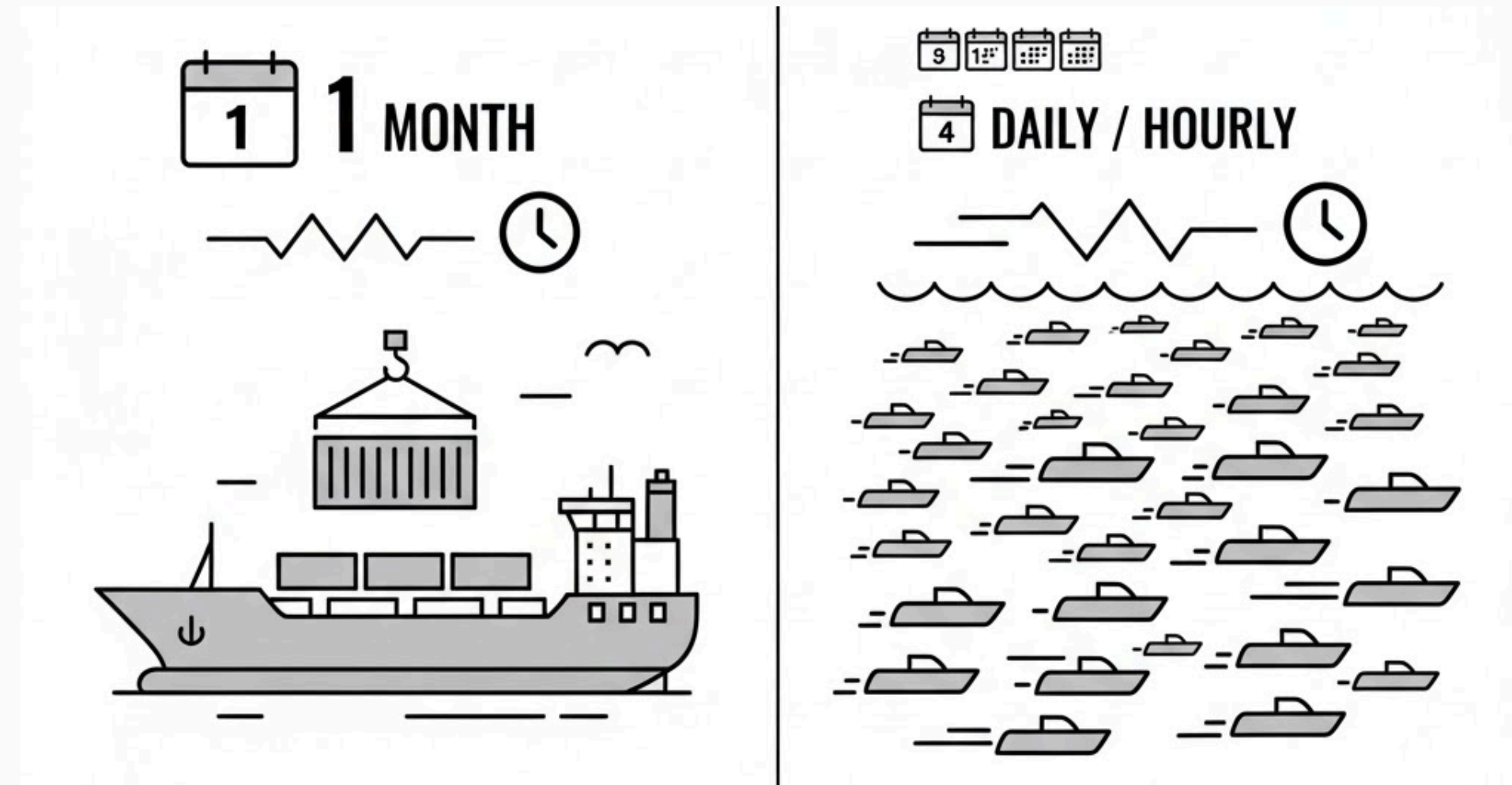
서비스의 트래픽이 잘 증가하지 않는 경우 : 모놀리스
특정 서비스가 자주, 과다하게 트래픽이 증가하는 경우: MSA

배포 빈도에 따른 결정



새로운 기능 추가가 거의 없는 경우 : 모놀리스
기능 추가도 잦고, 배포도 잦은 경우: MSA

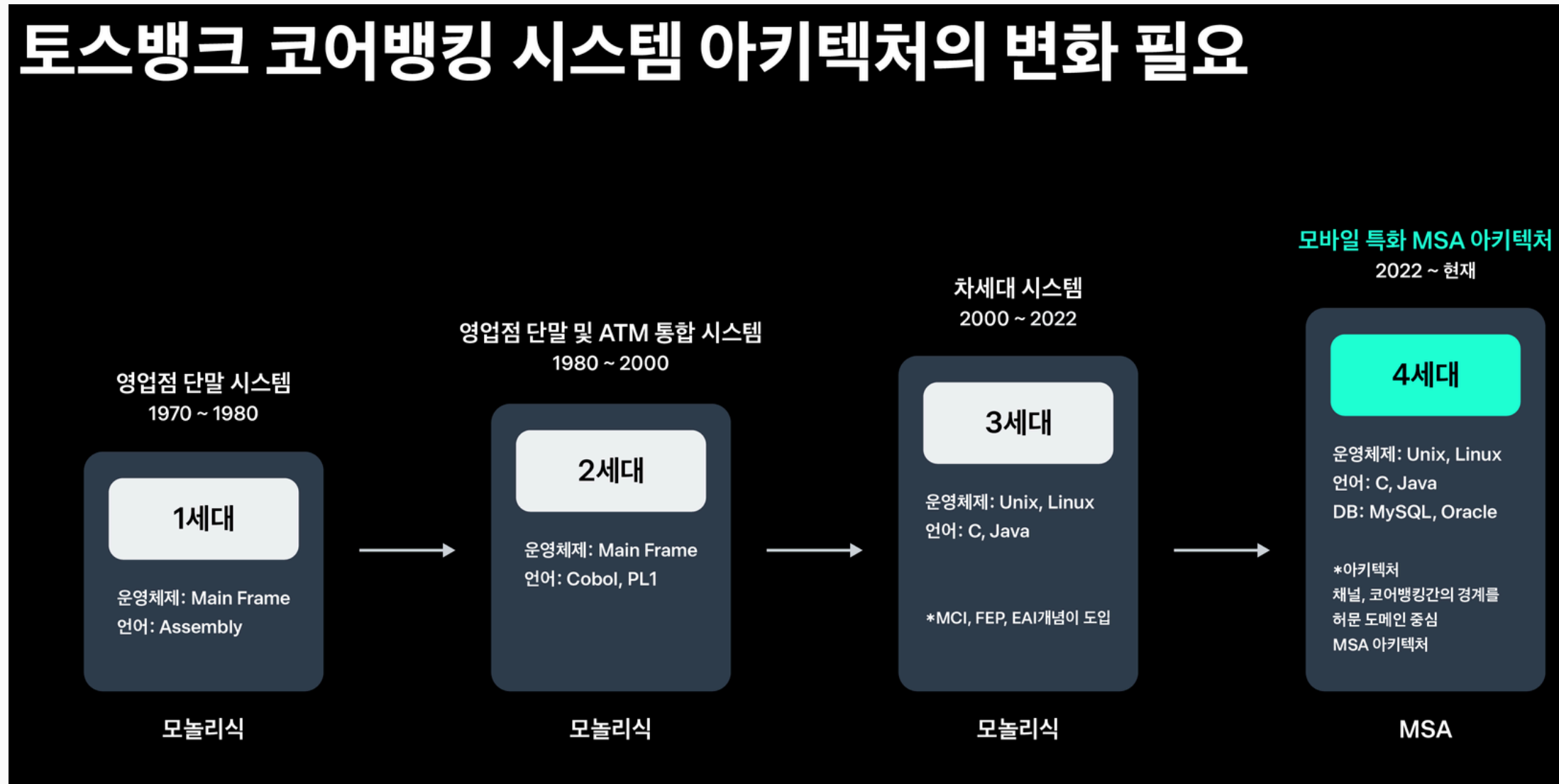
배포 빈도에 따른 결정



새로운 기능 추가가 거의 없는 경우 : 모놀리스
기능 추가도 잦고, 배포도 잦은 경우: MSA

MSA 적용사례

토스 코어 बैं킹 MSA 전환 사례



트래픽이 많아진 토스는 확장성을 위해
MSA로 변경했다.

넷플릭스가 주도하는 MSA

NETFLIX

넷플릭스는 약 3일간 서버가 중단된 적이 있다

이 외에 MSA를 도입한 회사들...



하나같이 거대한 회사들...

**팀이 5명이고, 빠르게 프로
토타입을 만들어야한다.
MSA를 사용하실래요?**



쿠폰기능에 갑자기 트래픽이 몰리면 MSA는 어떤 장점을 가질까요?



출처

<https://toss.tech/article/slash23-corebanking>

<https://github.com/eGovFramework/egovframe-common-components-msa-krds>

https://www.samsungsds.com/kr/insights/msa_and_netflix.html